

41. OOP Paradigm: What is Object-Oriented Programming (OOP)? How does it differ from procedural programming?

OOP is a programming paradigm based on the concept of "objects," which can contain data (attributes) and code (methods).

- Procedural Programming focuses on writing functions or procedures that operate on data (Top-down approach).
- OOP organizes programs around objects and data rather than logic and functions (Bottom-up approach), making it easier to manage large, complex projects.

42. Benefits of OOP: List and briefly explain at least three advantages.

- Modularity: Code is organized into independent classes/objects, making it easier to troubleshoot.
- Reusability: Through inheritance, you can use existing code in new programs.
- Maintenance: It is easier to update and maintain code because changes in one part of the system don't necessarily affect others.

43. Class vs. Object Analogy: Provide a real-world analogy.

Think of a Class as a Blueprint for a house. It defines the structure (number of windows, doors, rooms). An Object is the Actual House built from that blueprint. You can build multiple houses (objects) from a single blueprint (class).

44. Attributes vs. Methods: Explain using a Dog class.

- Attributes (Fields): The data/state of the object. Example for Dog: breed, color, age.
- Methods (Behaviors): The actions the object can perform. Example for Dog: bark(), eat(), sleep().

45. Encapsulation Definition: Why is it a fundamental principle?

Encapsulation is the technique of wrapping data (variables) and code (methods) together as a single unit. It is fundamental because it protects the data from outside interference and misuse by restricting direct access to the "innards" of an object.

46. Information Hiding: How is it achieved in Java?

Information hiding is the practice of concealing the internal details of a class. It is achieved using access modifiers (like private) to hide variables and providing controlled access through public getter and setter methods.

47. public vs. private: Explain the purpose and usage.

- private: Limits access to within the same class only. Used for data variables to ensure security.
- public: Allows access from any other class. Used for methods that need to be called by other parts of the program.

48. Getters and Setters: Why are they used? Illustrate for Student class.

Answer: They provide a safe way to view (get) and modify (set) private data.

Example:java

```
private String name;

public String getName() { return name; } // Getter

public void setName(String newName) { this.name = newName; } // Setter
```

49. Constructors: What is the purpose? When is it executed?

A constructor is a special method used to initialize objects. It is executed automatically at the moment an object is created using the `new` keyword.

50. Default vs. Parameterized Constructor: Can a class have both?

- Default Constructor: Has no arguments and provides default values to the object.
- Parameterized Constructor: Takes arguments to assign specific values to the object during creation.

Yes, a class can have both through a process called Constructor Overloading.

51. Car Class

```
class Car {
    private String make, model;
    private int year;
    private double speed;

    public Car(String make, String model, int year) {
        this.make = make; this.model = model; this.year = year; this.speed = 0;
    }
    public double getSpeed() { return speed; }
    public void accelerate(double inc) { speed += inc; }
    public void brake(double dec) { speed = Math.max(0, speed - dec); }
}
// In main: Car myCar = new Car("Toyota", "Camry", 2022); myCar.accelerate(20);
```

52. Student Class

```
class Student {
    private int studentId, age;
    private String name;
    private char grade;

    public Student(int id, String name, int age, char grade) {
        this.studentId = id; this.name = name; this.age = age; this.grade = grade;
    }
    public void setAge(int age) { this.age = age; }
    public void displayStudentInfo() {
        System.out.println(name + " (ID: " + studentId + ") - Grade: " + grade);
    }
}
```

53. BankAccount Class

```
class BankAccount {
    private String accountNumber, accountHolderName;
    private double balance;

    public BankAccount(String accNum, String name, double bal) {
        this.accountNumber = accNum; this.accountHolderName = name; this.balance = bal;
    }
    public void deposit(double amt) { if(amt > 0) balance += amt; }
    public void withdraw(double amt) {
        if(amt > 0 && amt <= balance) balance -= amt;
        else System.out.println("Invalid amount or Insufficient funds");
    }
}
```

54. Rectangle Class

```
class Rectangle {
    private double length, width;
    public Rectangle(double l, double w) { length = l; width = w; }
    public double calculateArea() { return length * width; }
    public double calculatePerimeter() { return 2 * (length + width); }
}
```

55. Dog Class with Multiple Constructors

```
class Dog {
    private String name;
```

```
private int age;

public Dog() { this.name = "Unknown"; this.age = 0; } // Default
public Dog(String name, int age) { this.name = name; this.age = age; } // Parameterized

public void bark() { System.out.println(name + " says Woof!"); }
}
```